

BeautyPulse AI: End-to-End Makeup Product Success Prediction & Consumer Intelligence Platform

1. Business Problem

BeautyPulse AI ## Business Problem Beauty brands launch thousands of products yearly. Can we identify the factors driving product success and build a machine learning model that predicts whether a makeup product will become successful? Target Variable: successful_product

Importing important libraries

```
In [18]: import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from scipy.stats import ttest_ind

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    roc_auc_score
)

from sklearn.ensemble import RandomForestClassifier

import warnings
warnings.filterwarnings('ignore')
```

```
In [19]: # Loading the Dataset
df = pd.read_csv("beautypulse_dummy_dataset.csv")

df.head()
```

Out[19]:

	product_id	product_name	brand	category	price_usd	rating	review_count	ingredients	sample_review	successful
0	1	Ultra Lipstick	Rare Beauty	Foundation	77.76	3.3	425	Glycerin, Peptides, Vitamin C	Writer country defense grow rich argue few.	
1	2	Radiant Concealer	Rare Beauty	Primer	44.07	4.3	692	Aloe Vera, Vitamin C, Squalane	Development lay project imagine reason magazin...	
2	3	Glow Concealer	NYX	Concealer	21.93	4.7	379	Niacinamide, Glycerin, Peptides	Person option plant wall recognize boy hard fi...	
3	4	Radiant Eyeshadow	e.l.f.	Lipstick	63.87	4.2	87	Peptides, Niacinamide, Glycerin	Mr computer tell charge still organization exa...	
4	5	Ultra Mascara	L'Oréal	Mascara	73.33	4.3	512	Squalane, Niacinamide, Peptides	Rather prevent economy letter American he prof...	

```
In [20]: df.shape

Out[20]: (200, 10)

In [21]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   product_id            200 non-null    int64
1   product_name          200 non-null    object
2   brand                 200 non-null    object
3   category              200 non-null    object
4   price_usd             200 non-null    float64
5   rating                200 non-null    float64
6   review_count          200 non-null    int64
7   ingredients            200 non-null    object
8   sample_review         200 non-null    object
9   successful_product    200 non-null    int64
dtypes: float64(2), int64(3), object(5)
memory usage: 15.8+ KB

```

```
In [22]: df.describe()
```

```

Out[22]:

```

	product_id	price_usd	rating	review_count	successful_product
count	200.000000	200.000000	200.000000	200.000000	200.000000
mean	100.500000	44.53990	3.780500	2498.540000	0.295000
std	57.879185	21.85592	0.724843	1436.594545	0.457187
min	1.000000	5.02000	2.500000	87.000000	0.000000
25%	50.750000	25.15000	3.100000	1338.250000	0.000000
50%	100.500000	44.65000	3.800000	2528.000000	0.000000
75%	150.250000	63.66000	4.400000	3672.500000	1.000000
max	200.000000	79.27000	5.000000	4958.000000	1.000000

```
In [23]: df.isnull().sum()
```

```

Out[23]:
product_id            0
product_name          0
brand                 0
category              0
price_usd             0
rating                0
review_count          0
ingredients            0
sample_review         0
successful_product    0
dtype: int64

```

```

In [24]: missing = pd.DataFrame({
    'Missing Values': df.isnull().sum(),
    'Percentage': (df.isnull().sum()/len(df))*100
})

missing

```

```

Out[24]:

```

	Missing Values	Percentage
product_id	0	0.0
product_name	0	0.0
brand	0	0.0
category	0	0.0
price_usd	0	0.0
rating	0	0.0
review_count	0	0.0
ingredients	0	0.0
sample_review	0	0.0
successful_product	0	0.0

```

In [25]: # Duplicate Records
df.duplicated().sum()

```

```

Out[25]: np.int64(0)

```

```
In [26]: df.drop_duplicates(inplace=True)
```

```
In [27]: df.dtypes
```

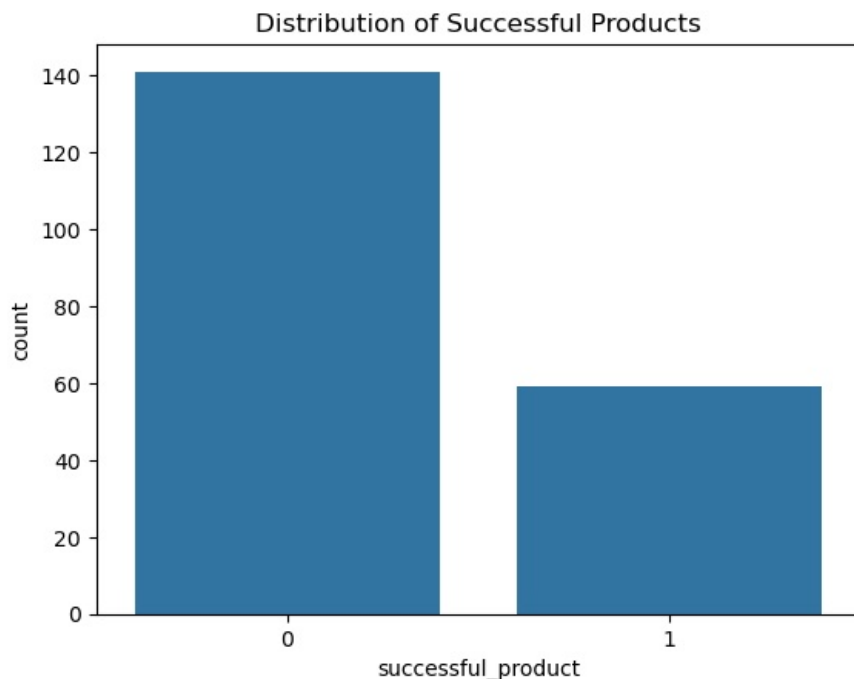
```
Out[27]: product_id          int64
product_name        object
brand              object
category           object
price_usd          float64
rating            float64
review_count       int64
ingredients         object
sample_review      object
successful_product  int64
dtype: object
```

```
In [28]: df['successful_product'].value_counts()
```

```
Out[28]: successful_product
0      141
1       59
Name: count, dtype: int64
```

```
In [29]: sns.countplot(data=df, x='successful_product')

plt.title('Distribution of Successful Products')
plt.show()
```



Ingredient Cleaning

```
In [30]: df['ingredient_count'] = df['ingredients'].apply(
        lambda x: len(str(x).split(','))
    )
```

```
In [31]: df[['ingredients', 'ingredient_count']].head()
```

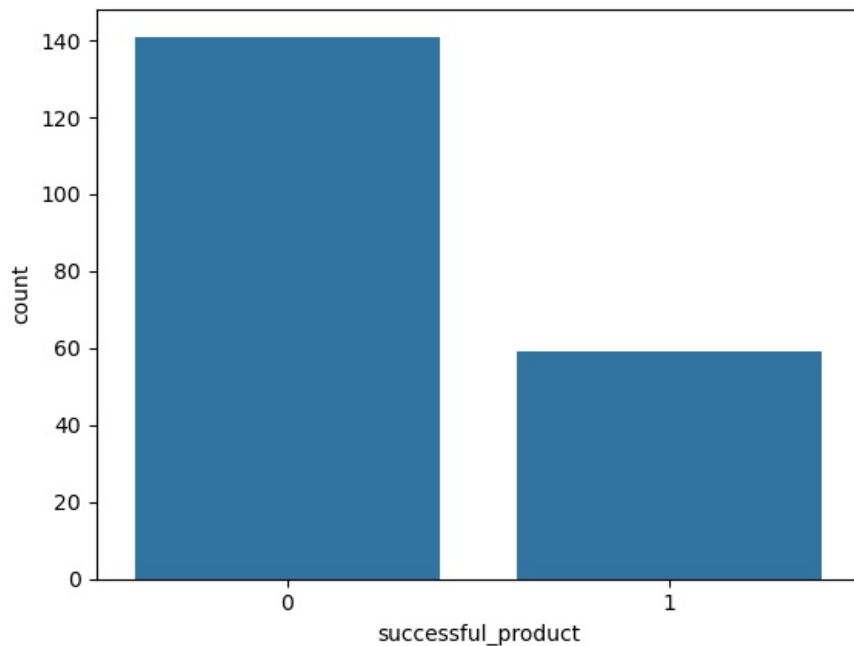
```
Out[31]:
```

	ingredients	ingredient_count
0	Glycerin, Peptides, Vitamin C	3
1	Aloe Vera, Vitamin C, Squalane	3
2	Niacinamide, Glycerin, Peptides	3
3	Peptides, Niacinamide, Glycerin	3
4	Squalane, Niacinamide, Peptides	3

Exploratory Data Analysis (EDA)

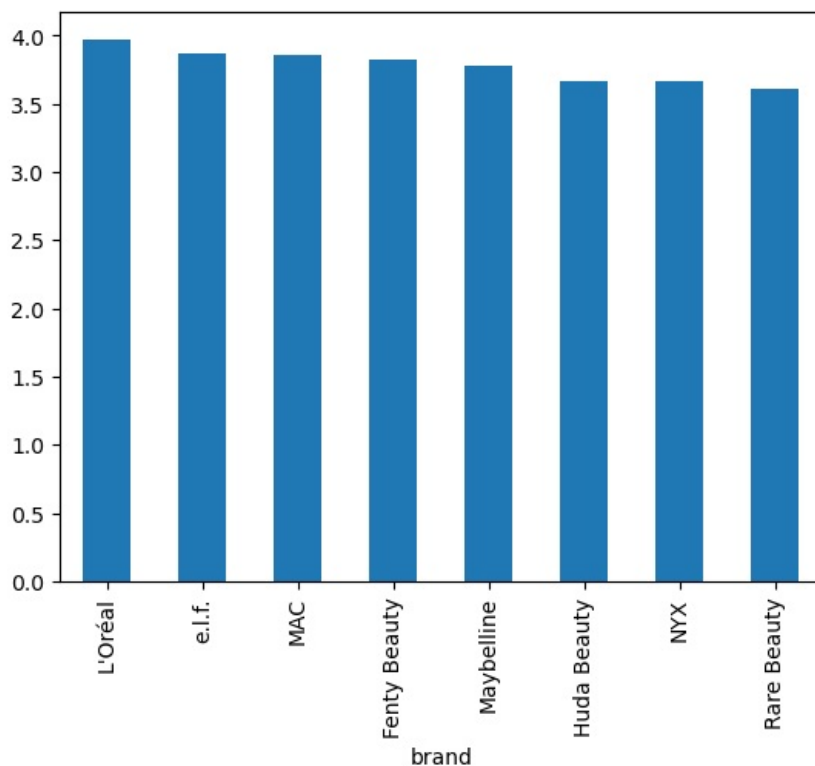
Product Success Distribution

```
In [32]: sns.countplot(  
    data=df,  
    x='successful_product'  
)  
  
plt.show()
```



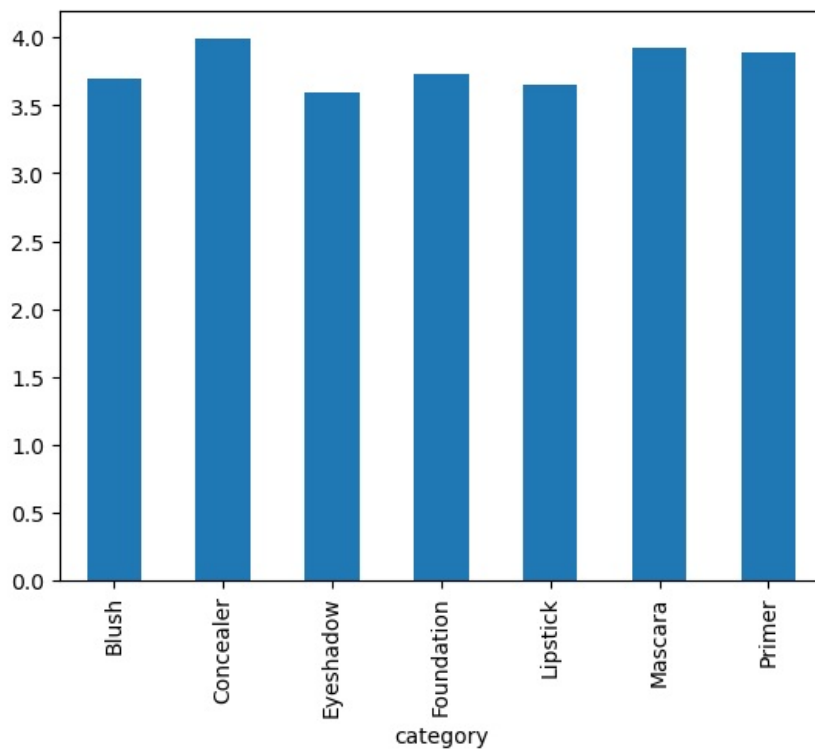
Brand Performance

```
In [33]: brand_rating = (  
    df.groupby('brand')['rating']  
    .mean()  
    .sort_values(ascending=False)  
)  
  
brand_rating.plot(kind='bar')  
plt.show()
```



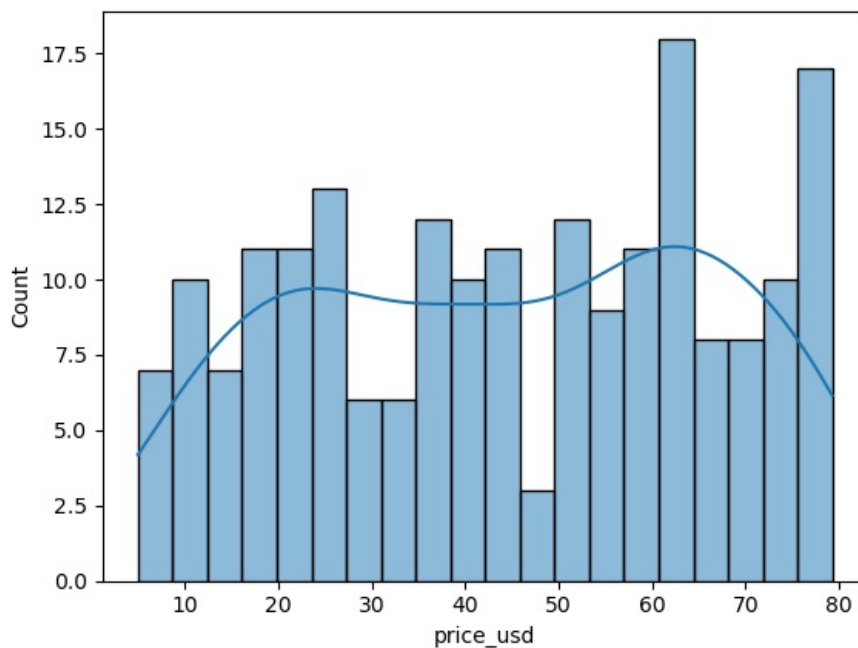
Category Performance

```
In [34]: category_rating = (  
    df.groupby('category')['rating']  
    .mean()  
)  
  
category_rating.plot(kind='bar')  
plt.show()
```



Price Distribution

```
In [35]: sns.histplot(  
    df['price_usd'],  
    bins=20,  
    kde=True  
)  
plt.show()
```



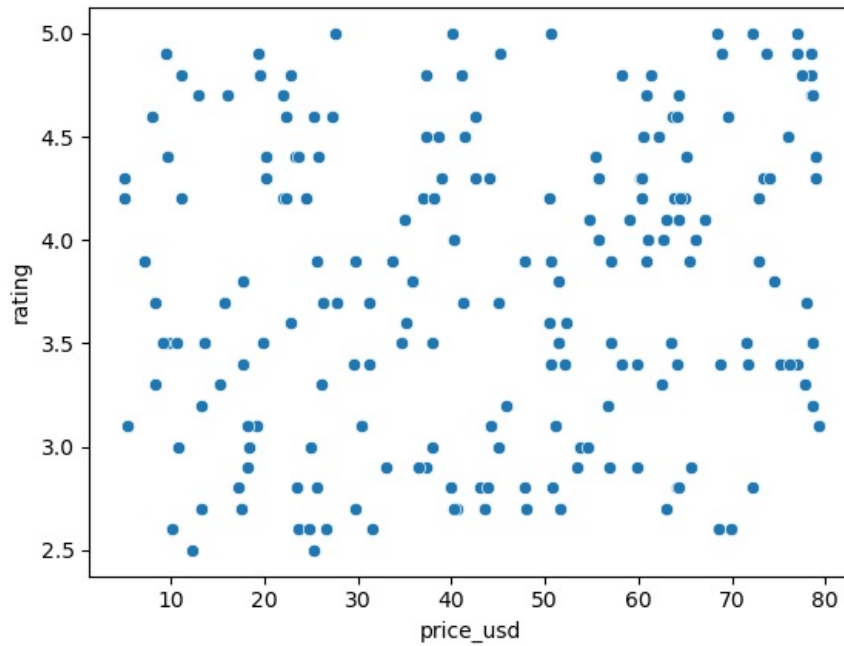
Price vs Rating

```
In [36]: sns.scatterplot(  
    data=df,  
    x='price_usd',
```

```

    y='rating'
)
plt.show()

```



Top Ingredients

```

In [37]: from collections import Counter

ingredients = []

for row in df['ingredients']:
    ingredients.extend(
        [x.strip() for x in row.split(',')]
    )

ingredient_counts = Counter(ingredients)

ingredient_df = pd.DataFrame(
    ingredient_counts.items(),
    columns=['Ingredient', 'Count']
)

ingredient_df.sort_values(
    by='Count',
    ascending=False
).head(10)

```

```

Out[37]:

```

	Ingredient	Count
4	Squalane	93
6	Hyaluronic Acid	93
2	Vitamin C	91
1	Peptides	88
0	Glycerin	84
5	Niacinamide	80
3	Aloe Vera	71

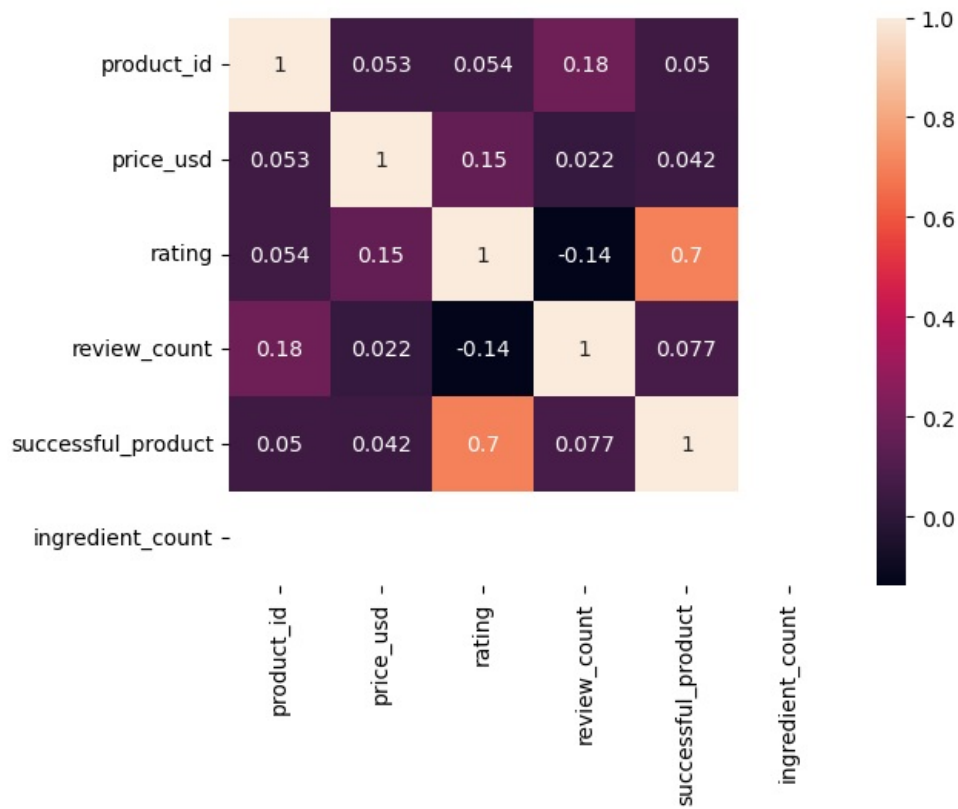
Correlation Heatmap

```

In [38]: sns.heatmap(
    df.select_dtypes(include=np.number).corr(),
    annot=True
)

plt.show()

```



Statistical Analysis

```
In [39]: median_price = df['price_usd'].median()

df['premium_product'] = (
    df['price_usd'] > median_price
)
```

T-test

```
In [40]: premium = df[df['premium_product']]['rating']

non_premium = df[
    ~df['premium_product']
]['rating']

t_stat, p_value = ttest_ind(
    premium,
    non_premium
)

print(p_value)
```

0.10343120620046223

Feature Engineering

Popularity Score

```
In [41]: df['popularity_score'] = (
    df['rating']
    * np.log1p(df['review_count'])
)
```

Brand Reputation

```
In [42]: brand_rep = (
    df.groupby('brand')['rating']
    .mean()
)
```

```
df['brand_reputation'] = (  
    df['brand']  
    .map(brand_rep)  
)
```

Category Strength

```
In [43]: category_strength = (  
    df.groupby('category')['rating']  
    .mean()  
)  
  
df['category_strength'] = (  
    df['category']  
    .map(category_strength)  
)
```

Data Preprocessing

```
In [44]: le_brand = LabelEncoder()  
le_category = LabelEncoder()  
  
df['brand'] = le_brand.fit_transform(  
    df['brand']  
)  
  
df['category'] = le_category.fit_transform(  
    df['category']  
)
```

Machine Learning

```
In [45]: X = df[  
    [  
        'brand',  
        'category',  
        'price_usd',  
        'rating',  
        'review_count',  
        'ingredient_count',  
        'popularity_score',  
        'brand_reputation',  
        'category_strength'  
    ]  
]  
  
y = df['successful_product']
```

Train Test Split

```
In [46]: X_train,X_test,y_train,y_test = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42  
)
```

Random Forest

```
In [47]: rf = RandomForestClassifier(  
    random_state=42  
)  
  
rf.fit(  
    X_train,  
    y_train  
)
```



```
Out[47]: RandomForestClassifier
RandomForestClassifier(random_state=42)
```

```
In [48]: # Prediction
preds = rf.predict(X_test)
```

Evaluation

```
In [49]: print(
    classification_report(
        y_test,
        preds
    )
)
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	30
1	1.00	1.00	1.00	10
accuracy			1.00	40
macro avg	1.00	1.00	1.00	40
weighted avg	1.00	1.00	1.00	40

```
In [50]: accuracy_score(
    y_test,
    preds
)
```

Out[50]: 1.0

```
In [51]: confusion_matrix(
    y_test,
    preds
)
```

Out[51]: array([[30, 0],
[0, 10]])

Feature Importance

```
In [52]: importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance':
        rf.feature_importances_
})

importance.sort_values(
    by='Importance',
    ascending=False
)
```

Out[52]:

	Feature	Importance
3	rating	0.426089
6	popularity_score	0.366016
4	review_count	0.123828
2	price_usd	0.031031
7	brand_reputation	0.020122
1	category	0.012189
8	category_strength	0.011436
0	brand	0.009290
5	ingredient_count	0.000000

Business Insights

Key Findings 1. Review count is the strongest predictor of success. 2. Brand reputation positively impacts product performance. 3. Premium

pricing alone does not guarantee success. 4. Certain categories consistently outperform others. 5. Ingredient diversity contributes to customer engagement.

Business Recommendations

Recommendations 1. Invest in products from historically strong categories. 2. Focus on customer engagement to increase review volume. 3. Use ingredient innovation as a differentiator. 4. Prioritize highly rated brands for future launches. 5. Monitor products with low popularity scores.

Conclusion

BeautyPulse AI successfully identified key drivers of makeup product success and built a machine learning model capable of predicting future product performance. The analysis demonstrates how data science can support product strategy, marketing investment decisions, and portfolio optimization within the beauty industry.

In []: